Now, visit http://localhost:3000/welcome in the browser: the page will hang as the request hits our byebug breakpoint. Back in the terminal, you should see a familiar byebug interface waiting for you:

```
=> Booting Puma
=> Rails 5.2.2 application starting in development
=> Run `rails server -h` for more startup options
Puma starting in single mode...
«...»
[2, 11] in /usr/src/app/app/controllers/welcome_controller.rb

    2:   def index
    3:     redis = Redis.new(host: "redis", port: 6379)
    4:     redis.incr "page hits"
    5:
    6:     @page_hits = redis.get "page hits"
    7:     byebug
=>  8:   end
    9: end
   10:
   11:
(byebug)
```

Feel free to experiment here to verify it works; for example, you could output the @page_hits variable to see its value. When you're ready to exit, press c (for "continue"), then Enter; the request will continue as normal, and the page should display in the browser. Press Ctrl-C to stop the web container.

Before moving on, remember to remove the byebug breakpoint from welcome_controller.rb, then start up the web server again:

```
$ docker-compose up -d web
```

Rails Server Not Starting?



If Rails fails to start because it thinks a server is already running, delete tmp/pids/server.pid on your local machine and try again. We'll see a better way to handle this in Chapter 9.

If you use IntelliJ or RubyMine, they have built-in support for using Ruby via docker-compose, including support for their debugger.[21]

Quick Recap

By now, using Docker should be starting to feel pretty familiar. In fact, much of this chapter was standard stuff we'd do in Rails—Docker got out of our way. Things got a little more tricky when it came to tests requiring JavaScript,

21. https://blog.jetbrains.com/ruby/2017/05/rubymine-2017-2-eap-1-docker-compose/